



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D4: Linux操作系统并发编程 之多进程

Linux下Web Server的设计

讲授内容

- 训练目的
- 训练原理
- 训练内容



- 训练目的

- 掌握UNIX/Linux系统创建进程的方法
- 掌握在代码中如何区别父子进程的方法
- 掌握父子进程之间的资源共享与异同
- 掌握等待子进程执行结束的方法
- 掌握在进程中执行另外一个可执行文件的方法



• 训练目的

- 了解生产者消费者问题中同步互斥的基本概念
- 掌握XSI信号量集机制
- 掌握XSI共享内存机制
- 掌握Linux中多进程协同工作的程序设计思想

训练原理：创建子进程

- 函数原型

- 头文件: `unistd.h`
- `pid_t fork(void);`

- 返回值

- **fork函数被正确调用后，将会在子进程中和父进程中分别返回！！**
 - 在子进程中返回值为0（不合法的PID，因为PID为0的进程是交换进程）
 - 在父进程中返回值为子进程ID（让父进程掌握其创建子进程的ID号）
- 出错返回-1



训练原理：创建子进程示例

```
int main(void){
    pid_t pid;
    pid=fork();
    if(pid== -1)
        printf( "fork error\n" );
    else if(pid==0){
        printf( "the returned value is %d\n" ,pid);
        printf( "in child process!!\n" );
        printf( "My PID is %d\n" ,getpid());}
    else{
        printf( "the returned value is %d\n" ,pid);
        printf( "in father process!!\n" );
        printf( "My PID is %d\n" ,getpid());}
    return 0;
}
```



```
wuxiaolin@wuxiaolin-virtual-machine: ~/LinuxC/Progress
wuxiaolin@wuxiaolin-virtual-machine:~/LinuxC/Progress$ ./progress
The returned value is 2547
In father process!!
My PID is 2546
The returned value is 0
In child process!!
My PID is 2547
```

父进程

子进程



训练原理：fork函数执行后父子进程的异同

• 父子进程相同

- 真实用户ID, 真实组ID
- 有效用户ID, 有效组ID
- 当前工作目录
- 文件模式掩码
- 信号量掩码
- 环境变量
- 堆
- 栈
- 文件表 (打开的文件)

■ 父子进程差异

- fork的返回值
- 进程ID
- 父进程ID
- 子进程的
tms_utime, tms_stime ,
tms_cutime, tms_ustime值
被设置为 0
- 文件锁



训练原理：fork的用法

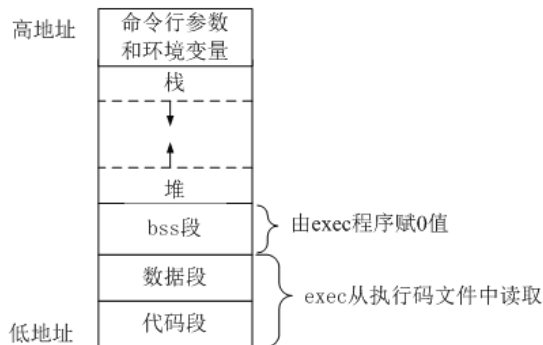
- 父进程希望复制自己（共享代码，拷贝数据空间），但使父子进程执行相同代码中的不同分支
 - 网络服务程序中，父进程等待客户端的服务请求，当请求达到时，父进程调用fork，使子进程处理该次请求，而父进程继续等待下一个服务请求到达
- 父子进程执行不同的可执行文件（父子进程具有完全不同的代码段和数据空间）
 - 子进程从fork返回后，立即**调用exec执行**另外一个可执行文件



训练原理：在进程中执行可执行文件

进程调用exec系列函数在进程中加载执行另外一个可执行文件

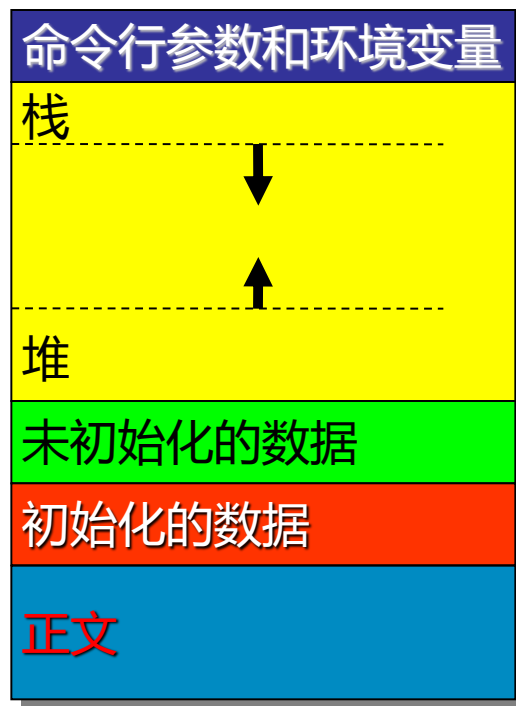
- exec系列函数**替换**了当前进程（执行该函数的进程）的正文段、数据段、堆和栈（来源于加载的可执行文件）
- 执行exec系列函数后从加载可执行文件的main函数开始重新执行
- exec系列函数并不创建新进程，所以在调用exec系列函数后其进程ID并未改变，已经打开的文件描述符不变



进程的资源



内核空间资源 (PCB)
由操作系统内核自动管理



用户空间资源
由用户程序或标准库管理



内核态用户态的两个示例

进程创建：当用户程序调用fork()时，内核会：

- ①分配新的 PCB，填充进程信息（如 PID、状态）。
- ②复制父进程的用户空间资源（代码段、数据段等）到子进程。
- ③返回子进程的 PID 给父进程，返回 0 给子进程。

文件读写：当用户程序调用read()时，内核会：

- 通过 PCB 查找文件描述符表
- 在内核空间读取文件内容。
- 将数据从内核缓冲区复制到用户空间的缓冲区。



训练原理: `exec1`函数

- 函数原型

- 头文件: `unistd.h`

- **`int exec1(const char *pathname, const char *arg0, .../*(char*)0*/);`**

- 参数

- `pathname`: 要执行程序的绝对路径名

- 可变参数: 要执行程序的命令行参数, 以 “`(char *)0`” 结束

- 返回值

- 出错返回-1

- **成功该函数不返回!**



训练原理: execl示例代码

```
int main(void)
{
    printf("entering main process---\n");
    if(fork()==0){
        execl("/bin/ls","ls","-l",NULL);
        printf("exiting main process ----\n");
    }
    return 0;
}
```

```
[zxy@test unixenv_c]$ cc execl.c
[zxy@test unixenv_c]$ ./a.out
entering main process---
total 104
-rwxrwxr-x. 1 zxy zxy      5976 Jul 12 22:54 a.out
-rw-r--r--. 1 zxy zxy      527 Jul 12 15:48 atexit02.c
-rw-r--r--. 1 zxy zxy      426 Jul 12 15:59 atexit03.c
-rw-r--r--. 1 zxy zxy      287 Jul 12 15:44 atexit.c
-rw-r--r--. 1 zxy zxy      472 Jul 10 12:39 creathole.c
```



训练原理：进程的终止

· 正常终止

- 从main返回
- 调用exit函数
- 调用_exit函数
- 最后一个线程从其启动例程中返回
- 最后一个线程调用pthread_exit

· 异常终止

- 调用abort，产生SIGABRT信号
- 接收到终止信号



训练原理：等待子进程终止

waitpid函数可以等待某个特定子进程终止

■ 函数原型

- 头文件：sys/wait.h

- **pid_t waitpid(pid_t pid, int *statloc, int options);**

■ 返回值

- 成功返回终止子进程ID，失败返回-1



训练原理：等待子进程终止

参数

- pid
 - $\text{pid} > 0$ ：等待进程ID等于pid的子进程执行终止
- statloc：存放子进程终止状态
- options：可以为0，也可以是WNOHANG（如果没有任何已经终止的子进程则马上返回，函数不等待，此时返回值为0）



训练原理：waitpid代码示例

```
pid_t pid_child, pid_return;

pid_child = fork();

if(pid_child < 0) printf("Error occurred on forking.\n");

else if(pid_child == 0){
    sleep(10);
    exit(0);}

do{

    pid_return = waitpid(pid_child,
                        NULL, WNOHANG);

    if(pid_return == 0){
        printf("No child exited\n");
        sleep(1);} }

while(pid_return == 0);

if(pid_return == pid_child)

    printf("successfully get child %d\n", pid_return);

else printf("some error occurred\n");
```

[illegible]

训练1

- 训练内容

- 在Linux中通过POSIX API（创建子进程，在进程中执行可执行文件(任意之前的训练内容即可)，等待子进程执行结束等）实现一个简单shell程序

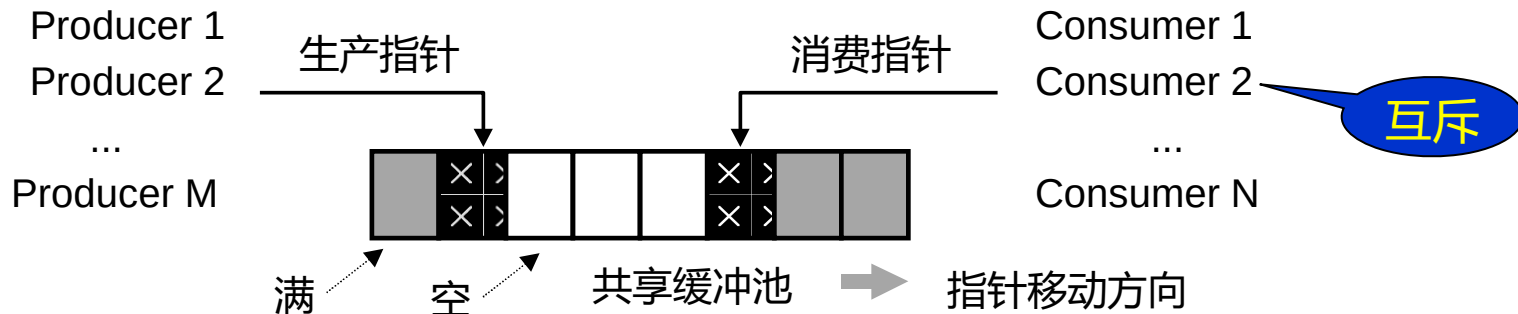


训练原理：任务的同步互斥

生产者/消费者问题

有缓冲区未被生产者使用时，生产者才能写入；有缓冲区已被生产者使用时，消费者才能读出——同步

问题描述：若干任务通过共享缓冲池（包含一定数量的缓冲区）交换数据。其中，“生产者”任务不断写入，而“消费者”任务不断读出；任何时刻只能有一个任务可对共享缓冲池进行操作。



训练原理：缓冲池的程序实现

- 缓冲池结构（5个缓冲区）

```
struct BufferPool
{
    char Buffer[5][100]; // 5个缓冲区
    int Index[5];
};
```

index: 缓冲区状态

- ①为0表示对应的缓冲区未被生产者使用，可分配但不可消费；
- ②为1表示对应的缓冲区已被生产者使用，不可分配但可消费

训练原理：生产者与消费者行为

- 生产者任务从源（文件，网络等）中读取数据后访问缓冲池，将数据放入未使用的缓冲区中（分配）
- 消费者任务访问缓冲池，从已被生产者使用的缓冲区中取走数据（消费），进行处理（计算、变换等）

训练原理：生产者与消费者之间的互斥

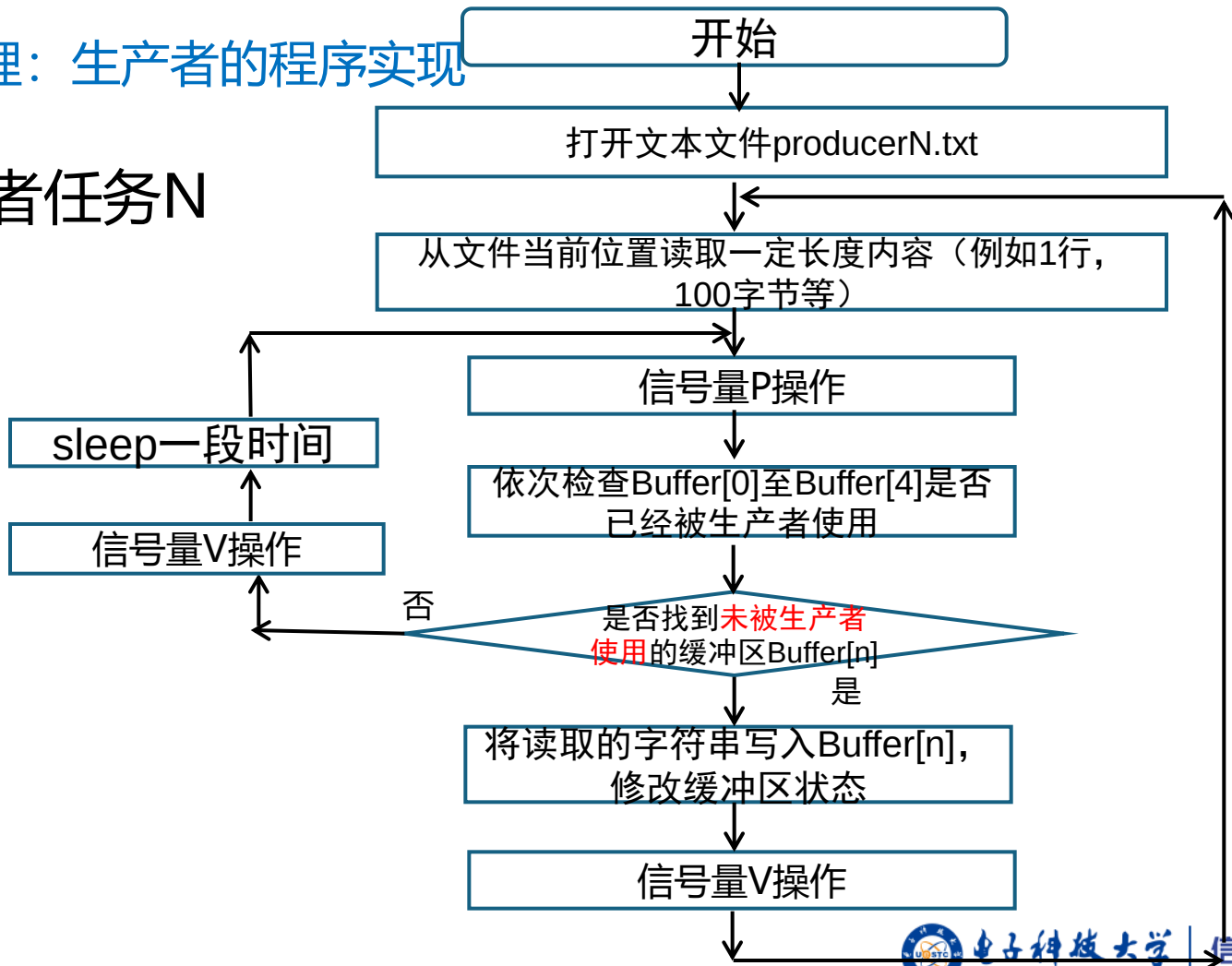
- 缓冲池整体作为临界资源
 - 当生产者任务正在访问缓冲池时，其他生产者和消费者任务不能访问缓冲池
 - 当消费者任务正在访问缓冲池时，其他生产者和消费者任务不能访问缓冲池
- 互斥机制：
 - 信号量集（包含1个信号量，初始值为1）
 - POSIX信号量

训练原理：生产者与消费者之间的同步

- 缓冲池中有缓冲区未被生产者使用时（缓冲池不满），生产者分配缓冲区
- 缓冲池中有缓冲区已被生产者使用时（缓冲池不空），消费者才能消费缓冲区
- 同步机制：
 - 通过缓冲池结构中的状态变量来指示
 - 通过信号量来指示未被使用缓冲区数量和已被使用缓冲区数量

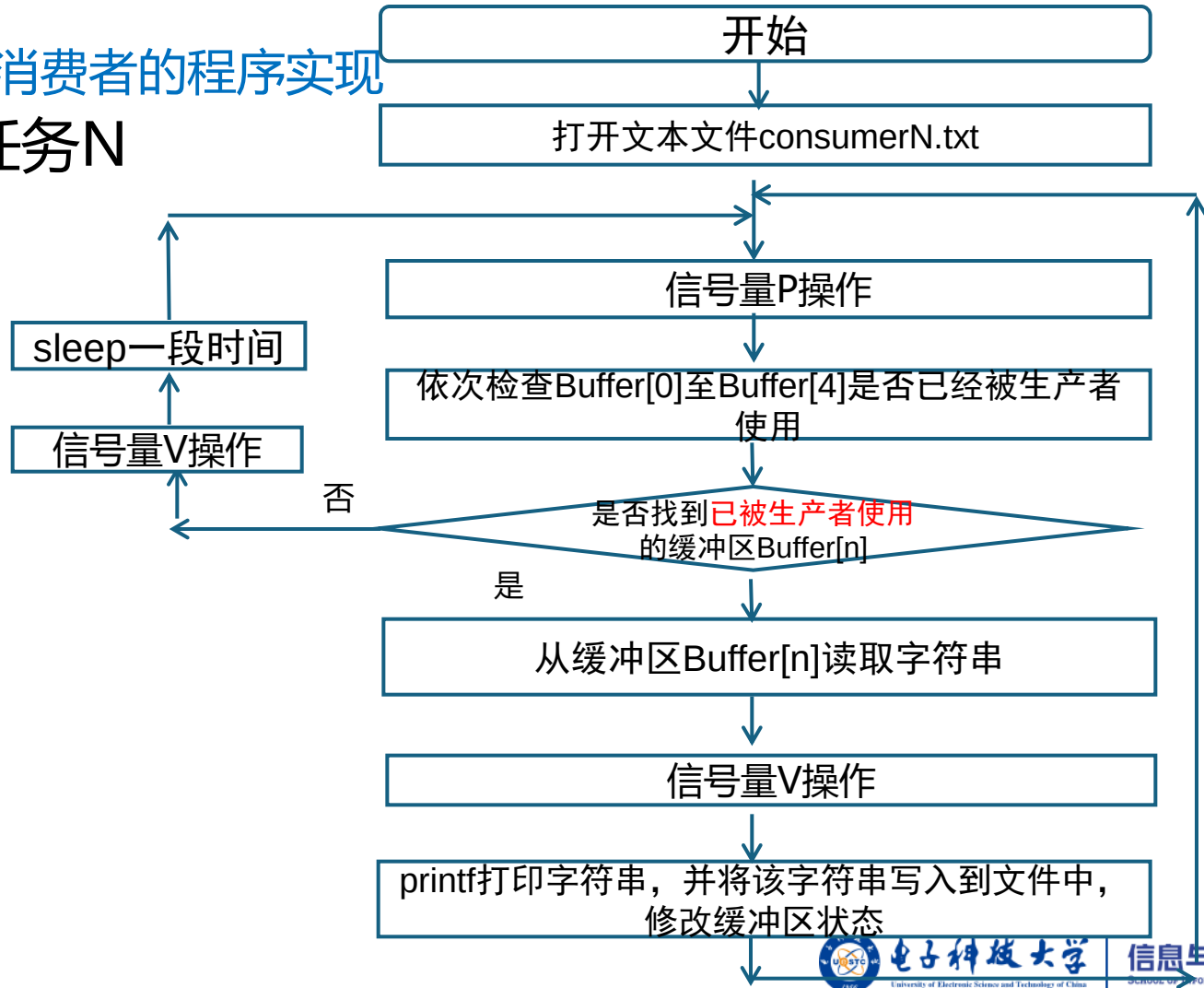
训练原理：生产者的程序实现

• 生产者任务N



训练原理：消费者的程序实现

• 消费者任务N



- 训练内容

- 综合运用所学知识在Linux系统上编写应用程序，要求至少支持2个生产者任务（进程）和2个消费者任务（进程）的同时运行
- 生产者任务和消费者任务之间通过**XSI共享内存机制**实现跨进程的共享缓冲池
- 生产者任务和消费者任务之间通过**XSI信号量集机制**实现对缓冲池的互斥访问
- 生产者与消费者之间的**同步**通过修改缓冲池结构中的**缓冲区状态变量来实现**

Linux编程技巧：实现多进程同时工作

- 分别编写、编译生产者程序和消费者程序
 - 在shell中分别执行生产者程序或消费者程序，每执行一次产生一个生产者任务（进程）或消费者任务（进程）
- 编写一个应用程序，通过fork创建子进程，在子进程中执行生产者的代码或消费者的代码

XSI IPC 系统调用

功能	信号量集	共享内存	消息队列
创建或打开一个IPC对象, 获得对IPC的访问权	semget	shmget	msgget
IPC操作: 发送/接收消息; 连接/释放共享内存; 信号量操作	semop	shmat/ shmdt	msgsnd/ msgrcv
IPC控制: 获得/修改IPC对象状态, “删除” IPC对象等	semctl	shmctl	msgctl

参考代码：消费者读取共享内存

```
struct shared_use_st
{
    int written;//非0：表示可读，0表示可写
    char text[TEXT_SZ];//记录写入和读取的文本};
int main(){
    int running = 1;
    void *shm = NULL;
    struct shared_use_st *shared;
    int shmid;
    shmid = shmget((key_t)1234, sizeof(struct
shared_use_st), 0666|IPC_CREAT);
    if(shmid == -1){
        exit(EXIT_FAILURE);}
    shm = shmat(shmid, 0, 0);
    if(shm == (void*)-1){
        exit(EXIT_FAILURE);}
    shared = (struct shared_use_st*)shm;
```

```
while(running){
    if(shared->written != 0)
    {
        printf("You wrote: %s", shared->text);
        sleep(rand() % 3);
        shared->written = 0;
        if(strncmp(shared->text, "end", 3) == 0)
            running = 0;
    }
    else
        sleep(1);
}
if(shmdt(shm) == -1){exit(EXIT_FAILURE);}
if(shmctl(shmid, IPC_RMID, 0) == -1)
{ exit(EXIT_FAILURE);}
exit(EXIT_SUCCESS);}
```



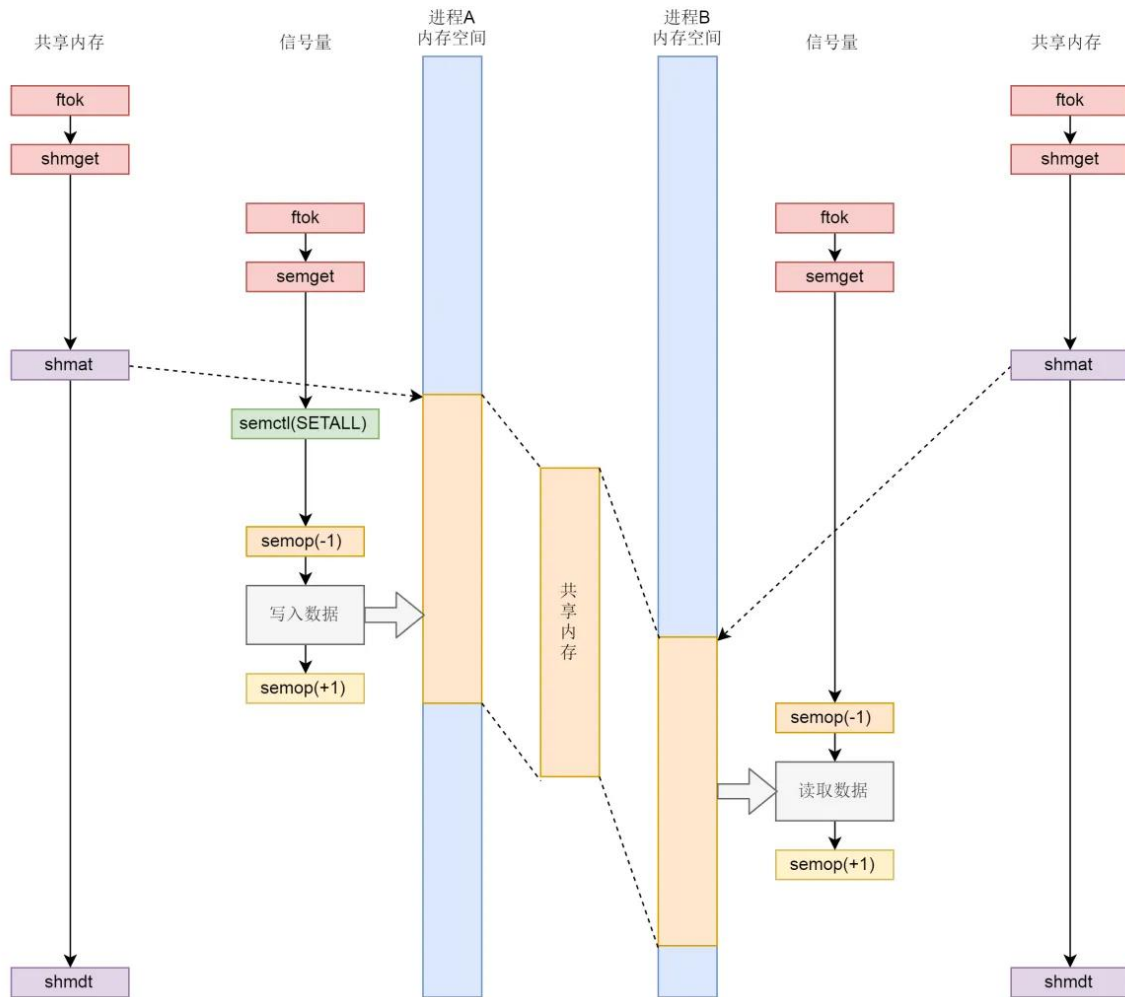
参考代码：生产者写共享内存

```
int main()
{
    int running = 1;
    void *shm = NULL;
    struct shared_use_st *shared = NULL;
    char buffer[BUFSIZ + 1];
    int shmid;
    shmid = shmget((key_t)1234, sizeof(struct
shared_use_st), 0666|IPC_CREAT);
    if(shmid == -1){
        exit(EXIT_FAILURE);}
    shm = shmat(shmid, (void*)0, 0);
    if(shm == (void*)-1){
        exit(EXIT_FAILURE);}
    printf("Memory attached at %X\n", (int)shm);
    shared = (struct shared_use_st*)shm;
```

```
while(running){
    if(shared->written == 1) {
        sleep(1);
        printf("Waiting...\n");}
    else{
        printf("Enter some text: ");
        fgets(buffer, BUFSIZ, stdin);
        strncpy(shared->text, buffer, TEXT_SZ);
        shared->written = 1; }
    if(strncmp(buffer, "end", 3) == 0)
        running = 0;
    }
    if(shmdt(shm) == -1) {exit(EXIT_FAILURE);}
    if(shmctl(shmid, IPC_RMID, 0) == -1)
        {exit(EXIT_FAILURE);}
    exit(EXIT_SUCCESS);}
```

producer

consumer



参考代码：创建或打开信号量集对象

```
#include<stdio.h>
#include<sys/sem.h>
#define MYKEY 0x1a0a
int main()
{
    int semid;
    semid=semget(MYKEY,1,IPC_CREAT|IPC_R | IPC_W| IPC_M);
    printf("semid=%d\n",semid);
    return 0;
}
```


参考代码：信号量设置初始值

```
int init_sem(int sem_id,int init_value)
{
    union semun sem_union;
    sem_union.val = init_value;
    if(semctl(sem_id,0,SETVAL,sem_union)==-1)
    {
        perror( "Initialize semaphore" );
        return -1;
    }
    return 1;
}
```

参考代码：信号量P操作

```
int semaphore_p(int sem_id, short sem_no)
{
    struct sembuf sem_b;
    sem_b.sem_num = sem_no; // 信号量集中信号量编号
    sem_b.sem_op = -1; // P操作，每次分配1个资源
    sem_b.sem_flg = SEM_UNDO;
    if (semop(sem_id, &sem_b, 1) == -1)
    {
        perror("semaphore_p failed");
        return 0;
    }
    return 1;
}
```

参考代码：信号量V操作

```
int semaphore_v (int sem_id, short sem_no)
{
    struct sembuf sem_b;
    sem_b.sem_num = sem_no; //信号量集中信号量编号
    sem_b.sem_op = 1; //V操作, 每次释放1个资源
    sem_b.sem_flg = SEM_UNDO;
    if (semop(sem_id, &sem_b, 1) == -1)
    {
        perror("semaphore_v failed");
        return 0;
    }
    return 1;
}
```

参考代码：删除信号量对象

```
int del_sem(int sem_id)
{
    union semun sem_union;
    if(semctl(sem_id,0,IPC_RMID,sem_union)==-1)
    {
        perror( "Delete semaphore" );
        return -1;
    }
    return 1;
}
```

Webserver V0.4版

多进程处理客户请求



mini_webserver

工程基础模块 Day1 && Day4

构建子模块 D1
Makefile
编译、清理、调试版本

配置子模块 D1
Config.c
端口、根目录、线程数

日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查

CSV存储子模块 D2
csv_store.c
用户数据加载和保存

索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

TCP监听子模块 D6
server.c
socket, bind, listen

连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

进程模型子模块 D4
process.c
父子进程协同

线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护

线程池子模块 D8
thread_pool.c
任务队列、工作线程

事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

请求解析子模块 D10
http_request.c
方法、URL、Header、Body

静态资源子模块 D11
static_file.c
HTML、CSS、图片读取

GET/POST处理子模块 D12
handler.c
表单提交、参数读取

路由配置子模块 D13
router.c
URL匹配、处理函数绑定

认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查

响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

每日增加功能的相关测试

如: test_day01.sh

如: bench.c 并发压力测试/响应时间等

Web Server 后面要同时处理多个客户端请求。V0.4版本用请求文件模拟客户端请求。父进程负责扫描请求文件并创建子进程，子进程负责处理一个请求，父进程等待所有子进程结束并记录结果。

新增文件包括：

- include/request_handler.h
- include/process_server.h
- src/request_handler.c
- src/process_server.c
- tests/test_day04.sh
- requests/hello.req
- requests/user_find.req
- requests/missing.req
- outputs/.gitkeep

修改文件包括

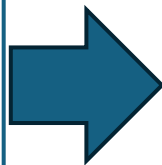
- src/main.c
- src/log.c
- include/log.h
- Makefile
- README.md
- docs/debug-log.txt (可选)

V0.4版 mini_webserver

requests/hello.req
GET /hello

requests/user_find.req
GET /users/zhangsan

requests/missing.req
GET /not-found



outputs/hello.out
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 12
Hello, Web!

outputs/user_find.out
FOUND zhangsan 123456 13800000000

outputs/missing.out
HTTP/1.1 404 Not Found



- process requests 能创建多个子进程
- 每个请求文件都有对应输出文件
- 父进程能等待所有子进程结束
- 日志中能看到父进程和子进程记录
- 解释：fork 后父子进程分别执行了哪段代码
- 解释：为什么要用 waitpid

